

Resumo

Este handout apresenta os fundamentos da linguagem JavaScript: sua origem, evolução histórica, sintaxe básica, mecanismos de funções, estruturas de dados essenciais e a forma como ela se integra a documentos HTML. O material foi organizado para acompanhar as aulas da disciplina e deve ser lido em conjunto com a prática constante de escrita de código.

Como usar este material: ao longo do handout você encontrará boxes coloridos com diferentes funções. Boxes verdes trazem dicas práticas de desenvolvimento. Boxes amarelos propõem exercícios de fixação. Boxes vermelhos alertam para armadilhas comuns. Boxes azuis apresentam informações contextuais importantes.

Universidade Federal da Fronteira Sul
Campus Chapecó

gian@uffs.edu.br

Versão: março de 2026

Keywords:

Sintaxe; Funções; Arrays; Objetos; Callbacks; Integração com HTML

1. Origem e Evolução do JavaScript

Para compreender por que o JavaScript possui determinadas características incluindo algumas que podem parecer inconsistentes é importante conhecer o contexto em que ele surgiu. A linguagem nasceu de uma necessidade prática e foi desenvolvida sob forte pressão de tempo, o que deixou marcas profundas na sua arquitetura original.

No início dos anos 1990, a Web era essencialmente estática: documentos HTML eram baixados do servidor e exibidos sem nenhuma interação adicional. Qualquer processamento exigia uma requisição ao servidor, o que tornava a experiência lenta e pouco responsiva, especialmente com as conexões dial-up da época.

A Netscape Communications, fabricante do navegador mais popular da época o Netscape Navigator identificou a necessidade de uma linguagem leve que pudesse ser interpretada diretamente pelo navegador, permitindo validar formulários, exibir alertas e responder a ações do usuário sem a necessidade de comunicação com o servidor.

Em 1995, Brendan Eich foi contratado pela Netscape com a missão inicial de incorporar a linguagem Scheme ao navegador. Por razões estratégicas, a empresa decidiu que a nova linguagem deveria ter uma sintaxe mais próxima do Java que era extremamente popular na época. O resultado foi criado em apenas dez dias e chamado inicialmente de Mocha, depois LiveScript, e finalmente JavaScript.

JavaScript não é Java. A semelhança no nome é puramente estratégica. Java é uma linguagem compilada, fortemente tipada e orientada a objetos de forma clássica. JavaScript é interpretada, dinamicamente tipada e baseada em protótipos. As duas linguagens compartilham influências sintáticas do C, mas são fundamentalmente diferentes em design e propósito.

1.1. ECMAScript

Em 1996, a Microsoft criou sua própria versão da linguagem, chamada JScript, para o Internet Explorer. A incompatibilidade entre as implementações da Netscape e da Microsoft gerou problemas sérios para os desenvolvedores, que precisavam escrever código diferente para cada navegador.

Para resolver esse problema, a Netscape submeteu o JavaScript à ECMA International (European Computer Manufacturers Association), um organismo de padronização. O resultado foi o padrão ECMAScript, publicado em 1997. A partir desse momento, “ECMAScript” passou a ser o nome oficial da especificação da linguagem, enquanto “JavaScript” ficou como o nome comercial utilizado pelos navegadores.

Hoje, quando falamos em “JavaScript moderno”, estamos nos referindo às versões mais recentes do ECMAScript. É comum abreviar o nome como “ES” seguido do número da versão: ES5, ES6, ES2020, e assim por diante.

1.2. ES6 (2015)

Entre 1999 (ES3) e 2009 (ES5), o desenvolvimento da especificação foi lento e marcado por conflitos internos no comitê técnico. A linguagem permaneceu relativamente estagnada enquanto o ecossistema web crescia exponencialmente.

O ES5, em 2009, trouxe melhorias importantes, mas foi o ES6 também chamado de ES2015 que representou a maior evolução já vista na linguagem. Publicado em junho de 2015 após anos de desenvolvimento, ele introduziu uma série de recursos que transformaram a forma de escrever JavaScript e tornaram a linguagem significativamente mais expressiva e segura.

Versão	Principais Contribuições
ES1 / ES2 (1997–1998)	Primeiras versões; especificação inicial da linguagem
ES3 (1999)	Expressões regulares, tratamento de exceções (<code>try/catch</code>), melhorias gerais
ES5 (2009)	Strict mode, métodos funcionais de array (<code>map</code> , <code>filter</code> , <code>reduce</code>), JSON nativo, getters e setters
ES6 / ES2015	<code>let</code> e <code>const</code> , arrow functions, classes, módulos (<code>import/export</code>), promises, template literals, desestruturação, spread operator, <code>Map</code> e <code>Set</code>
ES2017	<code>async/await</code> , <code>Object.entries()</code> , <code>Object.values()</code> , padding de strings
ES2018	Rest/spread para objetos, iteração assíncrona, <code>Promise.finally()</code>
ES2020	Optional chaining (<code>?.</code>), nullish coalescing (<code>??</code>), <code>Promise.allSettled()</code> , <code>globalThis</code>
ES2022+	Top-level <code>await</code> , métodos de array (<code>.at()</code>), <code>Object.hasOwn()</code> , melhorias em classes

Tabela 1. Evolução do ECMAScript e principais contribuições.

 **Compatibilidade e Transpilação.** Navegadores mais antigos podem não suportar os recursos mais recentes do ECMAScript. Ferramentas como o Babel fazem a transpilação: convertem código ES6+ para uma versão compatível com navegadores mais antigos. Em projetos profissionais, o Babel é configurado automaticamente por frameworks como React, Vue e Angular.

1.3. JavaScript Fora do Navegador

Por décadas, JavaScript foi uma linguagem exclusiva do navegador. Em 2009, Ryan Dahl criou o Node.js, um runtime que permite executar JavaScript no servidor, usando o motor V8 do Google Chrome. Isso transformou o JavaScript em uma linguagem de propósito geral, usada hoje tanto no front-end (navegador) quanto no back-end (servidor), além de ferramentas de linha de comando, automação e muito mais.

Ao longo desta disciplina, utilizaremos tanto o JavaScript no navegador quanto ambientes Node.js para executar código. Embora a sintaxe seja idêntica, algumas APIs diferem: o navegador oferece o DOM e o `fetch`, enquanto o Node.js oferece acesso ao sistema de arquivos e módulos próprios.

2. Sintaxe Básica

JavaScript herdou grande parte de sua sintaxe do C e do Java, o que torna a curva de aprendizado inicial mais suave para quem já conhece essas linguagens. No entanto, há características importantes que distinguem o JavaScript, especialmente no que diz respeito a tipos de dados, escopo de variáveis e coerção de tipos.

2.1. Comentários

Comentários são ignorados pelo interpretador e servem para documentar o código. Bons comentários explicam o porquê de uma decisão, não o que o código faz o código em si deve ser legível o suficiente para isso.

```
// Comentário de linha única

/*
  Comentário de múltiplas linhas.
  Útil para explicações mais longas ou para
  desabilitar blocos de código temporariamente.
*/

/**
 * Comentário JSDoc padrão para documentar funções e classes.
 * @param {string} nome - O nome da pessoa a ser saudada.
 * @returns {string} A mensagem de saudação.
 */
function saudar(nome) {
  return "Olá, " + nome + "!";
}
```

Código 1. Comentários em JavaScript

2.2. Variáveis e Escopo

Variáveis são contêineres para armazenar valores. Em JavaScript, a declaração de variáveis evoluiu ao longo do tempo, e compreender as diferenças entre as três palavras-chave disponíveis `var`, `let` e `const` é fundamental para escrever código correto e previsível.

var. Variáveis declaradas com `var` têm escopo de função: elas existem em toda a função onde foram declaradas, independentemente do bloco em que aparecem. Além disso, sofrem *hoisting* – são ‘elevadas’ ao topo da função antes da execução, com valor `undefined`. Esse comportamento frequentemente causa bugs sutis e difíceis de rastrear.

```
function exemploVar() {
  console.log(x); // undefined não gera erro por causa do hoisting!
  var x = 10;
  console.log(x); // 10

  if (true) {
    var y = 20; // y existe em TODA a função, não só neste bloco
  }
  console.log(y); // 20 y ainda existe aqui!
}
```

Código 2. Exemplo de utilização de `var` para declaração de variáveis em JavaScript.

⚠ Armadilha com var em laços. O comportamento de `var` em laços assíncronos é uma fonte clássica de bugs. Com `var`, todas as iterações compartilham a mesma variável. Com `let`, cada iteração tem sua própria variável. Prefira sempre `let` ou `const`.

```
// Problema com var
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100); // imprime 3, 3, 3 (não 0, 1, 2!)
}

// Solução com let
for (let j = 0; j < 3; j++) {
  setTimeout(() => console.log(j), 100); // imprime 0, 1, 2 (correto!)
}
```

let. Introduzido no ES6, `let` declara variáveis com escopo de bloco: elas existem apenas dentro do bloco (delimitado por chaves) em que foram declaradas. Use `let` quando o valor da variável precisar ser reatribuído ao longo da execução.

```
let contador = 0;

for (let i = 0; i < 5; i++) {
  contador += i;
  // i só existe dentro deste bloco
}

// console.log(i); // ReferenceError: i is not defined
```

Código 3. Exemplo de utilização de `let` para declaração de variáveis em JavaScript.

```
let temperatura = 22;
```

const. const também tem escopo de bloco, mas não permite que a variável seja reatribuída após a declaração. É importante entender que const protege a referência, não o valor em si: objetos e arrays declarados com const podem ter seus conteúdos modificados.

```
const PI = 3.14159;
// PI = 3; // TypeError: Assignment to constant variable.

const usuario = { nome: "Ana", idade: 21 };
usuario.nome = "Beatriz"; // PERMITIDO mudamos o conteúdo do objeto
// usuario = {};          // ERRO não podemos reatribuir a variável

const numeros = [1, 2, 3];
numeros.push(4);          // PERMITIDO mutação do array
// numeros = [5, 6];      // ERRO reatribuição não permitida
```

Código 4. Exemplo de utilização de const para declaração de variáveis em JavaScript.

💡 **Regra prática.** comece sempre com const. Se perceber que precisa reatribuir a variável, mude para let. Nunca use var em código novo. Essa disciplina evita uma classe inteira de bugs relacionados a escopo e mutação acidental.

2.3. Tipos de Dados

JavaScript é dinamicamente tipado: uma variável não tem tipo fixo, e sim o valor que ela contém em dado momento. O tipo de um valor pode ser verificado com o operador typeof. Há sete tipos primitivos e um tipo de referência (object), conforme Exemplo 5.

```
// string  seqüências de caracteres, com aspas simples, duplas
// ou template literals
let nome      = "Maria";
let sobrenome = 'Silva';
let mensagem  = `Olá, ${nome} ${sobrenome}!`; // template literal (ES6)

// number  inteiros e ponto flutuante (não há tipo separado)
let inteiro   = 42;
let decimal   = 3.14;
let negativo  = -7;
let cientifico = 1.5e6; // 1.500.000
let infinito   = Infinity;
let naoNumero  = NaN;   // Not a Number (resultado de operação inválida)

// boolean
let verdadeiro = true;
let falso     = false;

// null  ausência intencional de valor
let semValor  = null;

// undefined  variável declarada mas sem valor atribuído
let indefinido;
console.log(indefinido); // undefined

// symbol (ES6)  identificador único (uso avançado)
const id = Symbol("id");

// bigint (ES2020)  inteiros arbitrariamente grandes
const grandeNumero = 9007199254740993n;
```

Código 5. Tipos primitivos em JavaScript.

Verificando Tipos com typeof

```
console.log(typeof "texto"); // "string"
console.log(typeof 42);      // "number"
console.log(typeof true);    // "boolean"
console.log(typeof undefined); // "undefined"
```

Código 6. Uso de typeof para verificar tipos em JavaScript.

```
console.log(typeof null);           // "object"  bug histórico da linguagem!
console.log(typeof {});            // "object"
console.log(typeof []);            // "object"  arrays também são objetos
console.log(typeof function(){}); // "function"
```

⚠️ typeof null retorna "object". Este é um bug histórico do JavaScript que nunca foi corrigido por questões de compatibilidade. Para verificar se um valor é null, use === null diretamente:

```
const valor = null;
if (valor === null) {
  console.log("É null");
}
```

Coerção de Tipos (Type Coercion)

JavaScript realiza conversão automática de tipos (coerção) em diversas situações, o que pode causar resultados surpreendentes. Entender quando e como a coerção ocorre é essencial para evitar bugs.

```
// Coerção em operações aritméticas
console.log("5" - 2); // 3    string convertida para número
console.log("5" + 2); // "52" número convertido para string (concatenação!)
console.log("5" * 3); // 15   ambos convertidos para número
console.log(true + 1); // 2    true vale 1
console.log(false + 1); // 1   false vale 0
console.log(null + 1); // 1    null vale 0
console.log("" + 1); // "1"   concatenação com string vazia

// Coerção em comparações com ==
console.log(0 == false); // true
console.log("" == false); // true
console.log(null == undefined); // true
console.log(null == 0); // false (null só é igual a undefined com ==)

// Comparação estrita com === (sem coerção)
console.log(0 === false); // false
console.log("" === false); // false
```

Código 7. Exemplos de coerções de tipos em JavaScript.

💡 Use sempre === e !== para comparações. O operador == realiza coerção de tipos, tornando o resultado imprevisível para quem não conhece todas as regras. O operador === compara valor e tipo sem conversão, produzindo resultados sempre previsíveis. Faça disso um hábito.

Conversão Explícita

Quando precisar converter tipos, prefira fazê-lo de forma explícita, tornando a intenção clara para quem lê o código.

```
// String Number
const n1 = Number("42"); // 42
const n2 = parseInt("42px"); // 42 (para até o primeiro não-número)
const n3 = parseFloat("3.14"); // 3.14
const n4 = +"42"; // 42 (operador unário +)

// Number String
const s1 = String(42); // "42"
const s2 = (42).toString(); // "42"
const s3 = (3.14).toFixed(1); // "3.1" (arredonda e converte)

// Qualquer tipo Boolean
const b1 = Boolean(0); // false
const b2 = Boolean(""); // false
const b3 = Boolean(null); // false
const b4 = Boolean(undefined); // false
```

Código 8. Exemplos de conversão explícita de tipos em JavaScript.

```
const b5 = Boolean(NaN);    // false
// Tudo o mais é true (incluindo "0", [], {})
const b6 = Boolean("0");    // true!
```

2.4. Operadores

Operadores Aritméticos

```
let a = 10, b = 3;

console.log(a + b); // 13 soma
console.log(a - b); // 7 subtração
console.log(a * b); // 30 multiplicação
console.log(a / b); // 3.333... divisão (sempre float)
console.log(a % b); // 1 módulo (resto da divisão)
console.log(a ** b); // 1000 exponenciação (ES2016)

// Incremento e decremento
let x = 5;
console.log(x++); // 5 (retorna ANTES de incrementar)
console.log(x); // 6
console.log(++x); // 7 (incrementa ANTES de retornar)

// Atribuição composta
x += 10; // x = x + 10
x -= 3; // x = x - 3
x *= 2; // x = x * 2
x /= 4; // x = x / 4
x %= 3; // x = x % 3
```

Código 9. Conjunto de operadores aritméticos em JavaScript.

Operadores Lógicos e Curtocircuito

Os operadores lógicos && (E) e || (OU) não retornam apenas true ou false eles retornam um dos operandos originais. Esse comportamento, chamado de curtocircuito (*short-circuit evaluation*), é muito utilizado em JavaScript.

```
// && retorna o primeiro valor falsy OU o último valor se todos forem truthy
console.log(true && "olá"); // "olá"
console.log(false && "olá"); // false
console.log(null && "olá"); // null
console.log(1 && 2 && 3); // 3

// || retorna o primeiro valor truthy OU o último se todos forem falsy
console.log(false || "padrão"); // "padrão"
console.log("" || "padrão"); // "padrão"
console.log(null || undefined); // undefined
console.log("valor" || "padrão"); // "valor"

// Uso prático: valores padrão (antes do ES2020)
function saudar(nome) {
  nome = nome || "visitante"; // se nome for falsy, usa "visitante"
  return "Olá, " + nome;
}

// ?? (Nullish Coalescing ES2020): só usa o padrão se for null ou undefined
const config = null;
const valor = config ?? "padrão"; // "padrão"
const zero = 0 ?? "padrão"; // 0 (0 não é null nem undefined!)
```

Código 10. Operadores lógicos e curtocircuito em JavaScript.

2.5. Estruturas de Controle

Condicional if/else e operador ternário

```
const nota = 7.5;

if (nota >= 9) {
  console.log("Aprovado com distinção");
} else if (nota >= 7) {
  console.log("Aprovado");
}
```

Código 11. Exemplo de condicional if/else e operador ternário em JavaScript.

```

} else if (nota >= 5) {
  console.log("Recuperação");
} else {
  console.log("Reprovado");
}

// Operador ternário: versão compacta para condicionais simples
const situacao = nota >= 7 ? "Aprovado" : "Reprovado";
console.log(situacao);

```

switch

O switch compara um valor com múltiplos casos usando igualdade estrita (===). O break é necessário para evitar o fall-through (a execução continuar para o próximo caso).

```

const diaSemana = new Date().getDay(); // 0 = domingo, 6 = sábado

switch (diaSemana) {
  case 0:
    console.log("Domingo");
    break;
  case 6:
    console.log("Sábado");
    break;
  case 1:
  case 2:
  case 3:
  case 4:
  case 5:
    console.log("Dia útil");
    break;
  default:
    console.log("Dia inválido");
}

```

Código 12. Exemplo de uso do comando switch em JavaScript.

Laços de repetição

```

// for quando sabe o número de iterações
for (let i = 0; i < 5; i++) {
  console.log(i); // 0, 1, 2, 3, 4
}

// while quando a condição de parada não é baseada em contagem
let senha = "";
while (senha !== "1234") {
  senha = prompt("Digite a senha:");
}

// do...while garante ao menos uma execução
let tentativas = 0;
do {
  tentativas++;
  console.log("Tentativa " + tentativas);
} while (tentativas < 3);

// for...of itera sobre valores de um iterável (array, string, Map, Set...)
const frutas = ["maçã", "banana", "laranja"];
for (const fruta of frutas) {
  console.log(fruta);
}

// for...in itera sobre as chaves de um objeto
const pessoa = { nome: "Ana", idade: 22, cidade: "Chapecó" };
for (const chave in pessoa) {
  console.log(chave + ": " + pessoa[chave]);
}

// break e continue

```

Código 13. Exemplos de laços de repetição em JavaScript.

```
for (let i = 0; i < 10; i++) {
  if (i === 3) continue; // pula a iteração atual
  if (i === 7) break;    // encerra o laço
  console.log(i);        // 0, 1, 2, 4, 5, 6
}
```

💡 **Prefira for...of para arrays.** Em código moderno, for...of é mais legível que o for clássico para iterar arrays quando você não precisa do índice. Se precisar de índice e valor, use o método `entries()`:

```
for (const [i, v] of array.entries()) {
  ...
}
```

Tratamento de Exceções

Quando uma operação pode falhar (acesso a dados externos, conversão de tipos, parsing de JSON, etc.), use `try/catch` para lidar com o erro de forma controlada.

```
try {
  const dados = JSON.parse('texto inválido'); // lança SyntaxError
  console.log(dados);
} catch (erro) {
  console.error("Erro ao fazer parse:", erro.message);
} finally {
  // sempre executado, com ou sem erro
  console.log("Tentativa de parse concluída.");
}

// Lançando erros customizados
function dividir(a, b) {
  if (b === 0) {
    throw new Error("Divisão por zero não é permitida.");
  }
  return a / b;
}

try {
  console.log(dividir(10, 0));
} catch (e) {
  console.error(e.message); // "Divisão por zero não é permitida."
}
```

Código 14. Exemplos de tratamento de exceções em JavaScript.

3. Funções

Funções são o bloco de construção fundamental do JavaScript. Mais do que simples sub-rotinas, funções em JavaScript são valores de primeira classe (*first-class citizens*): podem ser atribuídas a variáveis, armazenadas em objetos e arrays, passadas como argumentos e retornadas por outras funções. Essa característica distingue profundamente o JavaScript de linguagens como Java e C, e é a base de muitos padrões avançados da linguagem.

3.1. Declaração de Função (Function Declaration)

A forma mais direta de criar uma função é através da declaração, usando a palavra-chave `function` seguida do nome, parâmetros e corpo.

```
function somar(a, b) {
  return a + b;
}

console.log(somar(3, 4)); // 7
console.log(somar(10));  // NaN b é undefined, 10 + undefined = NaN
```

Código 15. Exemplos de declaração de função em JavaScript.

Um aspecto importante das declarações de função é o *hoisting*: antes de executar o código, o interpretador JavaScript “eleva” todas as declarações de função para o topo

do escopo. Isso significa que você pode chamar uma função antes de sua declaração no código-fonte.

```
// Isso funciona por causa do hoisting:
console.log(quadrado(5)); // 25

function quadrado(x) {
  return x * x;
}
```

Código 16. Exemplo de *hoisting* com declaração de função em JavaScript.

Hoisting Embora o *hoisting* de declarações de função seja um comportamento válido, é considerada boa prática declarar as funções antes de usá-las, tornando o fluxo do código mais fácil de seguir.

3.2. Expressão de Função (Function Expression)

Uma função também pode ser criada como parte de uma expressão por exemplo, sendo atribuída a uma variável. Neste caso, a função pode ser anônima (sem nome) ou ter um nome (*Named Function Expression*, útil para recursão e depuração).

```
// Função anônima atribuída a uma variável
const multiplicar = function(a, b) {
  return a * b;
};

console.log(multiplicar(3, 4)); // 12

// Named Function Expression (útil para recursão e stack traces mais claros)
const fatorial = function calcFatorial(n) {
  if (n <= 1) return 1;
  return n * calcFatorial(n - 1); // pode referenciar o próprio nome
};

console.log(fatorial(5)); // 120
```

Código 17. Exemplos de expressão de função em JavaScript.

Diferente das declarações, expressões de função **não** sofrem *hoisting*: a variável é elevada, mas não o valor (a função). Tentar chamar a função antes da atribuição resultará em erro.

```
// Isso gera erro:
console.log(dobrar(5)); // TypeError: dobrar is not a function

const dobrar = function(x) {
  return x * 2;
};
```

Código 18. Exemplo de ausência de *hoisting* em expressão de função (erro).

3.3. Arrow Functions (ES6)

Introduzidas no ES6, as *arrow functions* oferecem uma sintaxe mais concisa e possuem uma diferença semântica importante em relação ao **this** (tratado na seção de objetos). São amplamente usadas em código moderno, especialmente como *callbacks*.

```
// Sintaxe básica: (parâmetros) => expressão
const dobrar = (x) => x * 2;

// Parênteses opcionais com um único parâmetro
const triplicar = x => x * 3;

// Sem parâmetros: parênteses obrigatórios
const saudacao = () => "Olá!";

// Com múltiplas instruções: chaves e return obrigatórios
const calcularImc = (peso, altura) => {
  const imc = peso / (altura * altura);
  return imc.toFixed(2);
};
```

Código 19. Exemplos de *arrow functions* em JavaScript.

```
};

// Retornando um objeto literal: envolva em parênteses para evitar ambiguidade
const criarUsuario = (nome, idade) => ({ nome, idade });
// Sem os parênteses, as chaves seriam interpretadas como início de bloco
```

💡 **Quando usar *arrow function* vs. *function* tradicional.** Use *arrow functions* para funções curtas, *callbacks* e quando quiser herdar o *this* do contexto externo (comum em métodos de objetos e classes). Use *function* tradicional quando precisar de um *this* próprio (ex.: métodos em objetos literais que referenciam o próprio objeto) ou quando a função precisa ser usada como *constructor*.

3.4. Parâmetros: padrão, rest e destruturação

Parâmetros com valor padrão (default parameters)

ES6 permite definir valores padrão para parâmetros diretamente na assinatura da função. O padrão é usado quando o argumento é *undefined* (não fornecido ou explicitamente *undefined*).

```
function criarConexao(host = "localhost", porta = 3000, protocolo = "http") {
  return `${protocolo}://${host}:${porta}`;
}

console.log(criarConexao()); // http://localhost:3000
console.log(criarConexao("meusite.com", 443, "https")); // https://meusite.com:443
console.log(criarConexao(undefined, 8080)); // http://localhost:8080
```

Código 20. Exemplo de parâmetros com valor padrão em JavaScript.

Rest parameters

O operador *rest* (...) captura todos os argumentos restantes em um array. Deve ser sempre o último parâmetro da função.

```
function somarTodos(descricao, ...numeros) {
  const total = numeros.reduce((acc, n) => acc + n, 0);
  return `${descricao}: ${total}`;
}

console.log(somarTodos("Soma", 1, 2, 3, 4, 5)); // Soma: 15
console.log(somarTodos("Total")); // Total: 0

// Diferente de arguments (objeto legado, não funciona em arrow functions)
function legado() {
  console.log(arguments); // object { '0': 1, '1': 2, ... }
}

const moderno = (...args) => {
  console.log(args); // array real [1, 2, ...]
};
```

Código 21. Exemplo de *rest parameters* em JavaScript.

Desestruturação em parâmetros

É possível desestruturar objetos e arrays diretamente nos parâmetros da função, tornando a extração de propriedades mais explícita e legível.

```
// Desestruturação de objeto
function exibirUsuario({ nome, idade, cidade = "Desconhecida" }) {
  console.log(`${nome}, ${idade} anos, ${cidade}`);
}

exibirUsuario({ nome: "Carlos", idade: 25, cidade: "Chapecó" });
// Carlos, 25 anos, Chapecó

// Desestruturação de array
function primeiro([a, b, ...resto]) {
  console.log(a, b, resto);
}
```

Código 22. Exemplos de desestruturação em parâmetros em JavaScript.

```
primeiro([1, 2, 3, 4, 5]); // 1 2 [3, 4, 5]
```

3.5. Closures

Uma *closure* é formada quando uma função “lembra” do ambiente léxico em que foi criada, mesmo depois que esse ambiente não está mais ativo. Em termos práticos: uma função interna que acessa variáveis da função externa forma uma *closure*.

```
function criarContador(inicio = 0) {
  let contagem = inicio; // variável no escopo externo

  return {
    incrementar: () => ++contagem,
    decrementar: () => --contagem,
    valor: () => contagem
  };
}

const contador = criarContador(10);
console.log(contador.incrementar()); // 11
console.log(contador.incrementar()); // 12
console.log(contador.decrementar()); // 11
console.log(contador.valor());       // 11

// Cada chamada cria seu próprio escopo independente
const outroContador = criarContador(0);
console.log(outroContador.valor()); // 0 independente de "contador"
```

Código 23. Exemplo de *closure* em JavaScript.

Por que closures importam? Closures são a base de padrões como módulos (encapsulamento de dados privados), memoização (*caching* de resultados), *currying* e funções parcialmente aplicadas. Entender closures é essencial para compreender o comportamento de código JavaScript mais avançado.

Para pensar: Implemente uma função `criarMultiplicador(fator)` que retorna uma nova função. A função retornada deve multiplicar qualquer número pelo fator fixado. Exemplo: `const dobrar = criarMultiplicador(2); dobrar(5)` deve retornar 10. O que torna isso possível? Que conceito está sendo usado?

3.6. Funções de Alta Ordem (Higher-Order Functions)

Uma função de alta ordem (*higher-order function*) é aquela que recebe outras funções como argumento ou que retorna uma função. Esse conceito, oriundo da programação funcional, é nativo do JavaScript e está presente em toda a API padrão da linguagem.

```
// Recebe uma função como argumento
function aplicar(valor, transformacao) {
  return transformacao(valor);
}

console.log(aplicar(5, x => x * 2)); // 10
console.log(aplicar("olá", s => s.toUpperCase())); // "OLÁ"

// Retorna uma função
function multiplicadorDe(fator) {
  return (numero) => numero * fator;
}

const dobrar = multiplicadorDe(2);
const triplicar = multiplicadorDe(3);

console.log(dobrar(7)); // 14
console.log(triplicar(7)); // 21
```

Código 24. Exemplos de funções de alta ordem em JavaScript.

```
// Compondo funções
function compor(f, g) {
  return (x) => f(g(x));
}

const dobrarESomar1 = compor(x => x + 1, x => x * 2);
console.log(dobrarESomar1(5)); // (5 * 2) + 1 = 11
```

4. Arrays e Objetos

Arrays e objetos são as principais estruturas de dados do JavaScript. Compreender como criá-los, manipulá-los e combiná-los eficientemente é essencial para qualquer aplicação real. No JavaScript moderno, ambos contam com uma sintaxe bastante expressiva.

4.1. Arrays

Arrays são listas ordenadas que podem conter valores de qualquer tipo. Ao contrário de linguagens como C ou Java, arrays em JavaScript são dinâmicos: crescem e encolhem automaticamente, e não há restrição de tipo nos elementos.

Criação e Acesso

```
// Criação com literal de array (forma preferida)
const frutas = ["maçã", "banana", "laranja"];

// Acesso por índice (base zero)
console.log(frutas[0]); // "maçã"
console.log(frutas[2]); // "laranja"
console.log(frutas[10]); // undefined (não gera erro)
console.log(frutas.length); // 3

// Acesso ao último elemento
console.log(frutas[frutas.length - 1]); // "laranja"
console.log(frutas.at(-1)); // "laranja" (ES2022, mais legível)

// Arrays com tipos mistos (possível, mas geralmente evitado)
const misto = [1, "dois", true, null, { chave: "valor" }, [3, 4]];
```

Código 25. Criação e acesso a elementos de arrays em JavaScript.

Métodos de mutação

Os métodos a seguir modificam o array original (mutação *in-place*). Conheça o comportamento de cada um para evitar surpresas.

```
const lista = [1, 2, 3];

// Adicionando elementos
lista.push(4, 5); // adiciona ao final [1, 2, 3, 4, 5]
lista.unshift(0); // adiciona ao início [0, 1, 2, 3, 4, 5]

// Removendo elementos
lista.pop(); // remove e retorna o último [0, 1, 2, 3, 4]
lista.shift(); // remove e retorna o primeiro [1, 2, 3, 4]

// splice: remove, substitui ou insere em qualquer posição
// splice(indice, qtd a remover, ...itens a inserir)
lista.splice(1, 0, 99); // insere 99 na posição 1 [1, 99, 2, 3, 4]
lista.splice(2, 1); // remove 1 elemento da posição 2 [1, 99, 3, 4]
lista.splice(1, 2, 10, 20); // substitui 2 a partir da pos. 1 [1, 10, 20, 4]

// Ordenando
const letras = ["c", "a", "b"];
letras.sort(); // ordena lexicograficamente: ["a", "b", "c"]

// CUIDADO: sort com números precisa de função comparadora
const nums = [10, 2, 5, 1, 20];
nums.sort(); // [1, 10, 2, 20, 5] errado! (ordem lexicográfica)
nums.sort((a, b) => a - b); // [1, 2, 5, 10, 20] correto!
```

Código 26. Métodos de mutação de arrays em JavaScript.

```
nums.sort((a, b) => b - a); // [20, 10, 5, 2, 1] decrescente
```

⚠️ sort() modifica o array original. Se precisar preservar o array original, crie uma cópia antes: `[...array].sort(...)` ou `array.slice().sort(...)`. Esta é uma armadilha comum para quem vem de outras linguagens.

Métodos funcionais de array (imutáveis)

Os métodos a seguir não modificam o array original eles retornam novos arrays ou valores. São amplamente utilizados no desenvolvimento moderno por tornarem o código mais previsível e testável.

```
const notas = [6.0, 8.5, 4.5, 9.0, 7.2, 5.8];

// map: aplica uma função a cada elemento e retorna um novo array
const arredondadas = notas.map(nota => Math.round(nota));
// [6, 9, 5, 9, 7, 6]

// filter: retorna novo array com elementos que satisfazem o predicado
const aprovados = notas.filter(nota => nota >= 7);
// [8.5, 9.0, 7.2]

// reduce: acumula um único valor a partir dos elementos
// reduce(fn(accumulator, elementoAtual), valorInicial)
const soma = notas.reduce((acc, nota) => acc + nota, 0); // 41.0
const media = soma / notas.length; // 6.833...

// find: retorna o PRIMEIRO elemento que satisfaz a condição (ou undefined)
const primeiraNota7 = notas.find(nota => nota >= 7); // 8.5

// findIndex: igual ao find, mas retorna o ÍNDICE
const indice = notas.findIndex(nota => nota < 5); // 2

// some: retorna true se PELO MENOS UM elemento satisfaz a condição
const temReprovado = notas.some(nota => nota < 5); // true

// every: retorna true se TODOS os elementos satisfazem a condição
const todosAprovados = notas.every(nota => nota >= 7); // false

// flat: achata arrays aninhados
const aninhado = [1, [2, 3], [4, [5, 6]]];
console.log(aninhado.flat()); // [1, 2, 3, 4, [5, 6]] 1 nível
console.log(aninhado.flat(2)); // [1, 2, 3, 4, 5, 6] 2 níveis

// flatMap: combina map + flat(1) em uma única operação
const frases = ["olá mundo", "JavaScript é legal"];
const palavras = frases.flatMap(frase => frase.split(" "));
// ["olá", "mundo", "JavaScript", "é", "legal"]
```

Código 27. Métodos funcionais de arrays em JavaScript.

💡 Encadeamento de métodos. Como `map`, `filter` e outros retornam arrays, é possível encadeá-los diretamente, criando *pipelines* de transformação legíveis:

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 4.0 },
  { nome: "Carla", nota: 9.2 },
  { nome: "Diego", nota: 6.8 },
];

const nomesAprovadosOrdenados = alunos
  .filter(a => a.nota >= 7)
  .sort((a, b) => b.nota - a.nota)
  .map(a => a.nome);
// ["Carla", "Ana"]
```

Para pensar: Dado um array de objetos representando produtos com as propriedades `nome`, `preco` e `quantidade`, escreva uma *pipeline* que: (1) filtre apenas produtos com estoque > 0, (2) calcule o valor total de cada produto (`preco × quantidade`), (3) retorne o valor total em estoque (soma de todos). Tente implementar sem usar laços `for`.

4.2. Objetos

Objetos são a estrutura de dados central do JavaScript. São coleções de pares chave-valor onde as chaves são strings (ou Symbols) e os valores podem ser qualquer tipo incluindo outras funções, que quando armazenadas em objetos são chamadas de métodos.

Criação e Acesso

```
// Literal de objeto (forma mais comum)
const produto = {
  nome: "Teclado Mecânico",
  preco: 349.90,
  disponivel: true,
  categorias: ["periférico", "input"],
  // Método: função como propriedade
  exibir() {
    return `${this.nome} R$ ${this.preco.toFixed(2)}`;
  }
};

// Acesso com notação de ponto
console.log(produto.nome); // "Teclado Mecânico"
console.log(produto.exibir()); // "Teclado Mecânico R$ 349.90"

// Acesso com notação de colchetes (útil quando a chave é dinâmica)
const campo = "preco";
console.log(produto[campo]); // 349.90

// Adicionando e removendo propriedades dinamicamente
produto.marca = "Keychron";
delete produto.disponivel;

// Verificando se uma propriedade existe
console.log("nome" in produto); // true
console.log(produto.hasOwnProperty("marca")); // true
console.log(Object.hasOwn(produto, "marca")); // true (ES2022, preferível)
```

Código 28. Criação e acesso a propriedades de objetos em JavaScript.

Criação de objetos com classes (ES6)

Para criar múltiplos objetos com a mesma estrutura, pode-se usar a sintaxe de classes introduzida no ES6 (açúcar sintático sobre o sistema de protótipos do JavaScript).

```
class Aluno {
  constructor(nome, matricula) {
    this.nome = nome;
    this.matricula = matricula;
    this.notas = [];
  }

  adicionarNota(nota) {
    this.notas.push(nota);
  }

  get media() {
    if (this.notas.length === 0) return 0;
    return this.notas.reduce((a, b) => a + b, 0) / this.notas.length;
  }

  toString() {
    return `[${this.matricula}] ${this.nome} Média: ${this.media.toFixed(1)}`;
  }
}
```

Código 29. Exemplo de classe ES6 em JavaScript.

```

    }
}

const aluno = new Aluno("Pedro", "2024001");
aluno.adicionarNota(8);
aluno.adicionarNota(7.5);
aluno.adicionarNota(9);
console.log(aluno.media); // 8.166...
console.log(aluno.toString()); // [2024001] Pedro Média: 8.2

```

Métodos úteis de Object

```

const config = { host: "localhost", porta: 5432, banco: "producao" };

// Obtendo chaves, valores e entradas
console.log(Object.keys(config));
// ["host", "porta", "banco"]
console.log(Object.values(config));
// ["localhost", 5432, "producao"]
console.log(Object.entries(config));
// [["host", "localhost"], ["porta", 5432], ["banco", "producao"]]

// Iterando com for...of + entries
for (const [chave, valor] of Object.entries(config)) {
    console.log(`${chave} = ${valor}`);
}

// Mesclando objetos com spread (mais legível que Object.assign)
const padrao = { porta: 3000, debug: false };
const personalizado = { porta: 8080, host: "meu-servidor" };
const final = { ...padrao, ...personalizado };
// { porta: 8080, debug: false, host: "meu-servidor" }

// Tornando objeto imutável
const constante = Object.freeze({ pi: 3.14, e: 2.71 });
constante.pi = 99; // silenciosamente ignorado (ou erro em strict mode)
console.log(constante.pi); // ainda 3.14

```

Código 30. Métodos úteis do objeto Object em JavaScript.

Desestruturação de objetos e arrays

Desestruturação é uma sintaxe do ES6 que permite extrair valores de objetos e arrays em variáveis individuais de forma concisa. É amplamente usada no desenvolvimento moderno, especialmente ao trabalhar com parâmetros de funções e respostas de APIs.

```

const usuario = {
    id: 1,
    nome: "Fernanda",
    endereco: { cidade: "Chapecó", estado: "SC" }
};

// Desestruturação básica
const { nome, id } = usuario;
console.log(nome); // "Fernanda"

// Renomeando variáveis na desestruturação
const { nome: nomeUsuario } = usuario;
console.log(nomeUsuario); // "Fernanda"

// Valores padrão
const { apelido = "Sem apelido" } = usuario;
console.log(apelido); // "Sem apelido"

// Desestruturação aninhada
const { endereco: { cidade } } = usuario;
console.log(cidade); // "Chapecó"

// Desestruturação de arrays
const cores = ["vermelho", "verde", "azul"];
const [primaria, secundaria] = cores;

```

Código 31. Exemplos de desestruturação de objetos e arrays em JavaScript.

```

console.log(primaria); // "vermelho"
console.log(secundaria); // "verde"

// Pulando elementos
const [, , terceira] = cores;
console.log(terceira); // "azul"

// Trocando valores com desestruturação
let x = 1, y = 2;
[x, y] = [y, x];
console.log(x, y); // 2 1

```

5. Callbacks

Um *callback* é uma função passada como argumento para outra função, com a expectativa de ser chamada em algum momento: imediatamente, após um evento, ou após a conclusão de uma operação assíncrona. *Callbacks* são o mecanismo original do JavaScript para composição de comportamentos e programação assíncrona.

5.1. O Modelo de Concorrência do JavaScript

Para entender *callbacks*, é preciso primeiro compreender como o JavaScript executa código. Diferente de linguagens como Java, o JavaScript possui apenas uma *thread* de execução ou seja, só pode executar uma instrução por vez. No entanto, é capaz de iniciar operações que demoram (como carregar um arquivo ou fazer uma requisição de rede) sem bloquear a *thread* principal.

Isso é possível graças ao *Event Loop* e à *Call Stack*. Operações assíncronas são delegadas ao ambiente (navegador ou Node.js). Quando concluem, o *callback* associado é colocado na fila de tarefas (*task queue*) e, quando a *Call Stack* está vazia, o *Event Loop* o move para execução.

```

console.log("1 início"); // executado imediatamente

setTimeout(() => {
  console.log("3 dentro do timeout"); // executado DEPOIS
}, 0); // mesmo com 0ms, o callback vai para a fila!

console.log("2 fim"); // executado imediatamente

// Saída:
// 1 início
// 2 fim
// 3 dentro do timeout

```

Código 32. Demonstração do comportamento assíncrono com `setTimeout` em JavaScript.

Event Loop em resumo A *Call Stack* executa código síncrono. Operações assíncronas (timers, I/O, eventos) são processadas pelo ambiente. Seus *callbacks* ficam na *task queue*. O *Event Loop* constantemente verifica: “a *Call Stack* está vazia? Então mova o próximo *callback* da fila para execução.” Isso garante que o JavaScript nunca bloqueie a interface do usuário.

5.2. Callbacks Síncronos

Callbacks síncronos são executados imediatamente pela função receptora. Os métodos de array `map`, `filter` e `forEach` são exemplos clássicos.

```

// forEach executa o callback para cada elemento, em ordem, de forma síncrona
const numeros = [1, 2, 3, 4, 5];
numeros.forEach(function(numero, indice, arrayOriginal) {
  console.log(`Índice ${indice}: ${numero}`);
});

// Callback customizado: separação de responsabilidades
function processarItens(itens, transformar, filtrar) {
  return itens.filter(filtrar).map(transformar);
}

```

Código 33. Exemplos de *callbacks* síncronos em JavaScript.

```

}

const resultado = processarItens(
  [1, 2, 3, 4, 5, 6],
  x => x * 10,           // callback de transformação
  x => x % 2 === 0       // callback de filtro (apenas pares)
);
// [20, 40, 60]

```

5.3. Callbacks Assíncronos

Callbacks assíncronos são invocados após a conclusão de uma operação que não é imediata. Os exemplos mais comuns incluem temporizadores (`setTimeout`, `setInterval`), eventos do DOM e operações de I/O no Node.js.

```

// setTimeout: executa uma vez após o atraso
const timer = setTimeout(() => {
  console.log("Executado após 2 segundos");
}, 2000);

// clearTimeout: cancela um timeout ainda pendente
clearTimeout(timer);

// setInterval: executa repetidamente no intervalo dado
let contador = 0;
const intervalo = setInterval(() => {
  contador++;
  console.log("Tick", contador);
  if (contador >= 5) {
    clearInterval(intervalo); // para após 5 execuções
  }
}, 1000);

```

Código 34. Exemplos de *callbacks* assíncronos com temporizadores em JavaScript.

Callbacks em eventos do DOM

No navegador, *callbacks* são associados a eventos através do método `addEventListener`. O *callback* recebe um objeto `Event` com informações sobre o que ocorreu.

```

const botao = document.getElementById("meu-botao");

botao.addEventListener("click", function(evento) {
  console.log("Botão clicado!");
  console.log(evento.target); // o elemento que foi clicado
  console.log(evento.clientX); // posição X do mouse
});

// Removendo event listener (precisa de referência à função)
function aoClicar(e) { console.log("Clicado"); }

botao.addEventListener("click", aoClicar);
botao.removeEventListener("click", aoClicar);

```

Código 35. Exemplo de *callbacks* em eventos do DOM em JavaScript.

5.4. Padrão de Callback com Tratamento de Erros

Em Node.js, consolidou-se o padrão de *callback* com erro como primeiro argumento (*Error-First Callback* ou *Node-style callback*). Se a operação falhou, o primeiro argumento é o objeto de erro; se teve sucesso, é `null`.

```

const fs = require("fs");

// Leitura assíncrona de arquivo no Node.js
fs.readFile("dados.txt", "utf8", function(erro, conteudo) {
  if (erro) {
    console.error("Erro ao ler arquivo:", erro.message);
    return; // importante: encerra a execução do callback
  }
  console.log("Conteúdo:", conteudo);
});

```

Código 36. Exemplo do padrão *error-first callback* no Node.js.

```
});  
  
console.log("Esta linha executa ANTES de ler o arquivo");
```

5.5. Callback Hell e Suas Consequências

Quando operações assíncronas dependem umas das outras, o encadeamento de *callbacks* resulta em código profundamente aninhado. Este padrão — chamado de *callback hell* ou *pyramid of doom* — torna o código difícil de ler, manter e testar.

```
// Callback hell: autenticação  buscar perfil  buscar posts  processar  
login(usuario, senha, function(erro, token) {  
  if (erro) return tratarErro(erro);  
  buscarPerfil(token, function(erro, perfil) {  
    if (erro) return tratarErro(erro);  
    buscarPosts(perfil.id, function(erro, posts) {  
      if (erro) return tratarErro(erro);  
      processarPosts(posts, function(erro, resultado) {  
        if (erro) return tratarErro(erro);  
        console.log(resultado);  
        // mais níveis...  
      });  
    });  
  });  
});
```

Código 37. Exemplo de *callback hell* em JavaScript.

Solução: Promises e `async/await` As *Promises* (ES6) e a sintaxe `async/await` (ES2017) surgiram como resposta ao *callback hell*, permitindo escrever código assíncrono de forma sequencial e legível. Esses mecanismos serão estudados em detalhe quando trabalharmos com requisições HTTP e integração com APIs. A familiaridade com *callbacks* é o pré-requisito essencial para compreender como *Promises* funcionam internamente.

Para pensar: Implemente uma função `esperar(ms, callback)` que executa o *callback* após `ms` milissegundos. Em seguida, use essa função para imprimir “um”, “dois”, “três” com 1 segundo de intervalo entre cada palavra — sem usar `setInterval`. Quantos níveis de aninhamento você precisou?

6. Ambientes

O JavaScript é uma linguagem única que pode ser executada em ambientes muito diferentes. Compreender essa dualidade é fundamental: o código que você escreve pode parecer idêntico nos dois contextos, mas as APIs disponíveis, os mecanismos de entrada/saída e o modelo de execução diferem de forma significativa.

Por que o mesmo código pode se comportar diferente? Imagine que você escreve `console.log(window)` no seu arquivo `.js`. No navegador, isso imprime o objeto `window` (que representa a janela do browser). No Node.js, gera um `ReferenceError`, pois `window` não existe. Da mesma forma, `require()` funciona no Node.js mas não no navegador sem um *bundler* como o Webpack ou Vite. Sempre saiba em qual ambiente seu código vai rodar.

7. O Console do Navegador (DevTools)

O console do navegador é a ferramenta mais imediata para experimentar JavaScript. Ele está embutido em todos os navegadores modernos e oferece um ambiente interativo onde você pode executar código, inspecionar variáveis, depurar erros e observar o comportamento de páginas web em tempo real.

Característica	Navegador	Node.js
Motor JS	V8 (Chrome), SpiderMonkey (Firefox), JavaScriptCore (Safari)	V8 (mesmo motor do Chrome)
Quem executa	O navegador web	O processo <code>node</code> no terminal
Acesso ao DOM	Sim (<code>document</code> , <code>window</code> , etc.)	Não (DOM não existe)
Acesso ao SO	Restrito (<i>sandbox</i> de segurança)	Completo (arquivos, rede, processos)
<code>console.log()</code>	Exibe no console do DevTools	Exibe no terminal (<code>stdout</code>)
Entrada do usuário	<code>prompt()</code> , eventos, formulários HTML	<code>process.stdin</code> , <code>readline</code> , bibliotecas
Módulos	ES Modules (<code>import/export</code>) nativo	CommonJS (<code>require</code>) e ES Modules
Uso típico	Interação com página, DOM, eventos	APIs, scripts, ferramentas CLI, <i>backend</i>

Tabela 2. Comparativo entre os ambientes de execução do JavaScript.

7.1. Abrindo o Console

Há três formas principais de abrir as ferramentas de desenvolvedor:

Sistema	Atalho	Resultado
Windows / Linux	F12	Abre as DevTools na última aba usada
Windows / Linux	Ctrl + Shift + I	Abre as DevTools (alternativa)
Windows / Linux	Ctrl + Shift + J	Abre as DevTools diretamente na aba Console
Mac	Cmd + Option + I	Abre as DevTools
Mac	Cmd + Option + J	Abre diretamente no Console
Qualquer	Botão direito > Inspeccionar	Abre inspecionando o elemento clicado

Tabela 3. Atalhos para abrir o console do navegador.

Após abrir as DevTools, clique na aba *Console*. Você verá um *prompt* interativo (geralmente representado por `>`) onde pode digitar e executar expressões JavaScript imediatamente. Pressione `Enter` para executar; `Shift+Enter` para inserir uma quebra de linha sem executar.

 **Abra o console em qualquer site.** Você pode abrir o console em qualquer página da web e executar JavaScript no contexto daquela página. Isso significa que você pode inspecionar e até modificar o conteúdo da página em tempo real. Por exemplo, tente abrir o console no Google e digitar: `document.title = 'Minha página'`

7.2. O Objeto console

O objeto `console` oferece vários métodos para exibir informações. Embora `console.log()` seja o mais usado, os outros métodos são igualmente importantes, pois muitos navegadores aplicam formatação visual diferente para cada nível de mensagem.

```
// console.log() saída padrão de informações
console.log("Mensagem simples");
console.log("Valor:", 42);
console.log("Múltiplos:", "a", "b", "c"); // a b c
console.log("Objeto:", { nome: "Ana", idade: 21 }); // exibe estrutura interativa

// Interpolação com template literals (mais legível)
const nome = "Carlos";
const nota = 8.5;
```

Código 38. Principais métodos do objeto `console`.

```

console.log("Aluno: " + nome + ", nota: " + nota); // concatenação
console.log(`Aluno: ${nome}, nota: ${nota}`);      // template literal (preferível)

// console.error()  exibe em vermelho, ideal para erros
console.error("Algo deu errado:", "mensagem de erro");

// console.warn()  exibe em amarelo, ideal para avisos
console.warn("Atenção: variável não inicializada");

// console.table()  exibe arrays e objetos como tabela formatada
const alunos = [
  { nome: "Ana",  nota: 9.0 },
  { nome: "Bruno", nota: 7.5 },
  { nome: "Carla", nota: 8.2 },
];
console.table(alunos);

// console.group() e console.groupEnd()  agrupam saídas relacionadas
console.group("Detalhes do usuário");
console.log("Nome: Ana");
console.log("Email: ana@uffs.edu.br");
console.groupEnd();

// console.time() e console.timeEnd()  medem tempo de execução
console.time("ordenacao");
const arr = [3, 1, 4, 1, 5, 9, 2, 6].sort((a, b) => a - b);
console.timeEnd("ordenacao"); // ex: ordenacao: 0.123ms

// console.count()  conta quantas vezes um ponto foi atingido
for (let i = 0; i < 3; i++) {
  console.count("iteracao"); // iteracao: 1, iteracao: 2, iteracao: 3
}

```

 **console.log() com múltiplos argumentos.** Para variáveis, use vírgula (não concatenação) para preservar a inspeção interativa do objeto:

```

const usuario = { nome: "Ana", ativo: true };
// Ruim: converte para string, perde interatividade
console.log("Usuario: " + usuario); // "Usuario: [object Object]"
// Bom: exibe o objeto interativo e expansível
console.log("Usuario:", usuario);
// Ainda melhor: use o nome da variável como label automático (ES2015+)
console.log({ usuario }); // { usuario: { nome: "Ana", ativo: true } }

```

7.3. Executando Código no Console

O console do navegador é um REPL completo (*Read-Eval-Print Loop*): você digita uma expressão, ela é avaliada e o resultado é impresso imediatamente.

```

// Expressões simples  o resultado aparece imediatamente abaixo
2 + 2                  // 4
"hello".toUpperCase() // "HELLO"
[1, 2, 3].map(x => x * 2) // [2, 4, 6]
typeof null           // "object"

// Declarando variáveis no console (persistem durante a sessão)
let contador = 0;
contador++;
console.log(contador); // 1

// Definindo funções e chamando-as
function saudar(nome) {
  return "Olá, " + nome + "!";
}
saudar("UFFS"); // "Olá, UFFS!"

```

Código 39. Exemplos de uso interativo do console do navegador.

```
// Acessando e manipulando o DOM da página atual
document.title // título da página
document.body.style.background = "lightyellow" // muda cor de fundo
document.querySelectorAll("a").length // conta links na página

// Usando $ como atalho para querySelector (disponível no console)
$("h1") // primeiro h1 da página (atalho do DevTools)
$$("p") // todos os parágrafos (atalho para querySelectorAll)
```

O underscore _ no console No console do navegador, o caractere `_` armazena o resultado da última expressão avaliada. Isso é útil para encadear operações sem precisar guardar o resultado em uma variável explicitamente:

```
[5, 3, 8, 1, 9].sort((a, b) => a - b)
// [1, 3, 5, 8, 9]
_.reverse()
// [9, 8, 5, 3, 1]
```

7.4. Carregando um Arquivo .js no Navegador

O console é ótimo para experimentos rápidos, mas para código mais longo o fluxo normal é escrever um arquivo `.js` e carregá-lo em uma página HTML. Há duas formas principais: *inline* (código dentro da tag `script`) e externo (arquivo separado referenciado pelo atributo `src`).

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <title>Meu Programa</title>
</head>
<body>
  <h1>Testando JavaScript</h1>
  <!-- Script externo recomendado para código real -->
  <!-- defer garante que o DOM esteja pronto antes de executar -->
  <script src="app.js" defer></script>
</body>
</html>
```

Código 40. Carregando um arquivo `.js` externo em uma página HTML.

```
// app.js
console.log("Arquivo carregado com sucesso!");
const titulo = document.querySelector("h1");
console.log("Título da página:", titulo.textContent);
```

Código 41. Arquivo `app.js` referenciado pela página HTML.

 **Use Live Server no VS Code.** A extensão *Live Server* do VS Code cria um servidor local e recarrega o navegador automaticamente ao salvar o arquivo. Instale a extensão, clique com o botão direito no arquivo HTML e escolha *Open with Live Server*. Isso elimina a necessidade de recarregar manualmente o navegador a cada alteração.

7.5. Entrada de Usuário no Navegador

O navegador oferece três funções nativas para obter dados do usuário de forma síncrona, por meio de caixas de diálogo. São úteis para prototipação e aprendizado, mas em aplicações reais são substituídas por formulários HTML e eventos de teclado.

`prompt()` — *leitura de texto*

Exibe uma caixa de diálogo com uma mensagem e um campo de texto. Retorna a *string* digitada pelo usuário, ou `null` se o usuário cancelar. A execução do script fica suspensa até o usuário responder.

```
// Sintaxe: prompt(mensagem, valorPadrao)
const nome = prompt("Qual é o seu nome?");
console.log(nome); // string digitada, ou null se cancelado

// Com valor padrão pré-preenchido
const cidade = prompt("De que cidade você é?", "Chapecó");

// Tratando o cancelamento e a conversão de tipo
const entrada = prompt("Digite um número:");
if (entrada === null) {
  console.log("Usuário cancelou.");
} else {
  const numero = Number(entrada);
  if (isNaN(numero)) {
    console.log("Valor inválido não é um número.");
  } else {
    console.log("Número ao quadrado:", numero ** 2);
  }
}

// IMPORTANTE: prompt() sempre retorna string ou null
const idade = prompt("Sua idade:");
console.log(typeof idade); // "string"
console.log(typeof Number(idade)); // "number" após conversão
```

Código 42. Uso de `prompt()` para leitura de entrada do usuário.

`confirm()` — leitura booleana

Exibe uma caixa de diálogo com uma mensagem e dois botões: OK e Cancelar. Retorna `true` se o usuário clicar em OK, `false` se clicar em Cancelar ou fechar.

```
const confirmado = confirm("Deseja continuar?");
console.log(confirmado); // true ou false

// Uso prático: confirmação antes de ação destrutiva
const excluir = confirm("Tem certeza que deseja excluir este registro?");
if (excluir) {
  console.log("Registro excluído.");
} else {
  console.log("Operação cancelada.");
}
```

Código 43. Uso de `confirm()` para confirmação de ações.

`alert()` — exibição de mensagem

Exibe uma caixa de diálogo com uma mensagem e apenas o botão OK. Não lê entrada serve apenas para notificar o usuário.

```
alert("Operação concluída com sucesso!");
alert("Erro: campo obrigatório não preenchido.");
```

Código 44. Uso de `alert()` para notificações ao usuário.

⚠ Limitações de `prompt()`, `confirm()` e `alert()`. Essas funções bloqueiam completamente a execução do script e a interface do navegador até o usuário responder. Muitos navegadores as suprimem em *iframes* ou após múltiplas chamadas. Além disso, não funcionam em ambiente Node.js são APIs exclusivas do navegador. Em aplicações reais, use sempre formulários HTML e eventos do DOM.

Quando usar: apenas em exemplos de aula, prototipação rápida e exercícios de lógica onde a interface não é o foco. Nunca em aplicações reais.

Entrada via Formulário HTML (abordagem correta)

Em aplicações reais, a entrada do usuário é feita por meio de elementos HTML como `<input>`, `<textarea>` e `<select>`, combinados com eventos JavaScript. Essa abordagem é não-bloqueante, customizável e acessível.

```
// JavaScript (app.js)
const campo = document.getElementById("campo-nome");
const botao = document.getElementById("btn-saudar");
const resultado = document.getElementById("resultado");
```

Código 45. Leitura de entrada via formulário HTML e eventos JavaScript.

```
botao.addEventListener("click", () => {
  const nome = campo.value.trim();
  if (!nome) {
    resultado.textContent = "Por favor, digite seu nome.";
    return;
  }
  resultado.textContent = "Olá, " + nome + "!";
});

// Também responder ao Enter no campo
campo.addEventListener("keydown", (evento) => {
  if (evento.key === "Enter") {
    botao.click(); // simula o clique no botão
  }
});
```

Para pensar e implementar: Crie uma página HTML com dois campos numéricos (numero1 e numero2), quatro botões (somar, subtrair, multiplicar, dividir) e um parágrafo para exibir o resultado. Implemente a lógica em JavaScript. Trate o caso de divisão por zero e o caso de entradas não numéricas. Não use `prompt()`.

8. Node.js no Terminal

O Node.js permite executar JavaScript diretamente no terminal, sem necessidade de um navegador. Isso abre um universo de possibilidades: scripts de automação, ferramentas de linha de comando, servidores web, processamento de arquivos e muito mais.

8.1. Instalando e Verificando o Node.js

O Node.js pode ser baixado em <https://nodejs.org>. Recomenda-se instalar a versão LTS (*Long Term Support*), que é mais estável para uso em produção. A instalação inclui também o npm (*Node Package Manager*).

```
# Verificar se o Node.js está instalado e a versão
node --version    # ex: v20.11.0

# Verificar a versão do npm
npm --version     # ex: 10.2.4

# Linux (Ubuntu/Debian) instalação via nodesource:
# curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
# sudo apt-get install -y nodejs
```

Código 46. Verificando a instalação do Node.js e do npm.

 **Use o NVM para gerenciar versões do Node.js.** O NVM (*Node Version Manager*) permite instalar e alternar entre múltiplas versões do Node.js facilmente. É especialmente útil quando projetos diferentes exigem versões diferentes. Instale em: <https://github.com/nvm-sh/nvm>

8.2. Executando um Arquivo .js

A forma mais direta de executar JavaScript com Node.js é passar o caminho do arquivo como argumento para o comando `node`.

```
# Execute no terminal
node ola.js

# Execute a partir de qualquer diretório
node scripts/processar.js
node /home/usuario/projetos/api/index.js
```

Código 47. Executando arquivos JavaScript com o Node.js.

```
# Variáveis de ambiente podem ser passadas antes do comando
PORT=3000 node index.js           # Linux/Mac
set PORT=3000 && node index.js     # Windows CMD
$env:PORT=3000; node index.js      # Windows PowerShell

// ola.js
const hoje = new Date();
const hora = hoje.getHours();
let saudacao;

if (hora < 12) {
  saudacao = "Bom dia";
} else if (hora < 18) {
  saudacao = "Boa tarde";
} else {
  saudacao = "Boa noite";
}

console.log(saudacao + "! São " + hoje.toLocaleTimeString() + ".");
```

Código 48. Exemplo de script Node.js com saudação baseada no horário.

8.3. O REPL do Node.js

Assim como o console do navegador, o Node.js oferece um REPL interativo que pode ser ativado simplesmente digitando `node` no terminal sem argumentos.

```
# Abrir o REPL do Node.js
node

# O prompt muda para > indicando que você está no REPL
> 2 + 2
4
> "hello".toUpperCase()
'HELLO'
> [1, 2, 3].map(x => x ** 2)
[ 1, 4, 9 ]
> const fs = require('fs')
undefined
> fs.readdirSync('.')
[ 'index.js', 'package.json', 'node_modules' ]

# Comandos especiais do REPL (começam com .)
.help           # lista todos os comandos
.exit           # encerra o REPL (ou Ctrl+C duas vezes, ou Ctrl+D)
.clear          # limpa o contexto atual
.editor         # modo editor para código multilinha (encerre com Ctrl+D)
.load arq.js    # carrega e executa um arquivo no contexto atual
.save arq.js    # salva o histórico de comandos em um arquivo
```

Código 49. Usando o REPL interativo do Node.js.

O REPL do Node vs console do navegador Ambos são REPLs de JavaScript, mas com APIs diferentes. No REPL do Node, você tem acesso a `require()`, `process`, `fs` e outros módulos do Node.js, mas não tem `document`, `window` ou `fetch`. No console do navegador é o contrário. O `_` funciona da mesma forma nos dois: armazena o último resultado.

8.4. Saída no Terminal: `console.log()` e `process.stdout`

No Node.js, `console.log()` escreve na saída padrão (`stdout`) com uma quebra de linha ao final. Para casos mais específicos — como escrever sem quebra de linha ou escrever na saída de erro — usa-se `process.stdout` e `process.stderr` diretamente.

```
// console.log() saída com quebra de linha (\n ao final)
console.log("Linha 1");
console.log("Linha 2");

// console.error() escreve em stderr (saída de erro padrão)
```

Código 50. Saída no terminal com `console.log()` e `process.stdout`.

```
console.error("Erro crítico:", "arquivo não encontrado");

// process.stdout.write() escreve sem quebra de linha automática
process.stdout.write("Processando");
process.stdout.write(".");
process.stdout.write(".");
process.stdout.write(". Concluído!\n");
// Saída: Processando... Concluído!

// Uso prático: barra de progresso simples
for (let i = 0; i <= 10; i++) {
  process.stdout.write("\rProgresso: " + (i * 10) + "%");
  // \r (carriage return) volta ao início da linha sem pular linha
}
```

8.5. Leitura de Entradas no Node.js

Diferente do navegador, o Node.js não tem `prompt()` ou interface gráfica. A leitura de dados do usuário no terminal é feita de forma assíncrona, por meio de *streams*. O módulo `readline`, nativo do Node.js, é a ferramenta padrão para isso.

Argumentos de Linha de Comando (`process.argv`)

A forma mais simples de passar dados para um script Node.js é por meio de argumentos na linha de comando. Eles ficam disponíveis no array `process.argv`.

```
// saudacao.js executar com: node saudacao.js Ana 25
// process.argv é sempre um array:
// [0] = caminho do executável node
// [1] = caminho do arquivo sendo executado
// [2]+ = argumentos fornecidos pelo usuário

const args = process.argv.slice(2); // remove os dois primeiros elementos

if (args.length < 2) {
  console.error("Uso: node saudacao.js <nome> <idade>");
  process.exit(1); // encerra com código de erro (não-zero = erro)
}

const nome = args[0];
const idade = parseInt(args[1]);

if (isNaN(idade)) {
  console.error("Erro: a idade deve ser um número inteiro.");
  process.exit(1);
}

console.log("Olá, " + nome + "! Você tem " + idade + " anos.");
console.log("Em 10 anos, você terá " + (idade + 10) + " anos.");
```

Código 51. Lendo argumentos de linha de comando com `process.argv`.

 **process.exit()** e os códigos de saída. Por convenção Unix, um processo que termina com *exit code* 0 indica sucesso; qualquer valor diferente de 0 indica erro. Ferramentas de automação, CI/CD e scripts shell usam esse código para saber se o programa falhou. Sempre chame `process.exit(1)` (ou qualquer valor não-zero) ao encerrar por causa de um erro.

Módulo `readline` — Leitura Interativa

O módulo `readline` permite criar uma interface de leitura de linha a linha a partir de qualquer *stream* de entrada. Para leitura interativa no terminal, usamos `process.stdin` como entrada e `process.stdout` como saída.

```
// pergunta-simples.js
const readline = require('readline');

// Cria a interface de leitura ligada ao terminal
const rl = readline.createInterface({
```

Código 52. Leitura interativa com o módulo `readline`.

```

    input: process.stdin, // entrada = teclado do usuário
    output: process.stdout // saída = terminal
  });

  // rl.question(pergunta, callback) exibe pergunta e aguarda resposta
  rl.question("Qual é o seu nome? ", (nome) => {
    console.log("Olá, " + nome + "! Bem-vindo.");
    // IMPORTANTE: feche a interface ao terminar
    // Sem isso, o processo Node.js fica esperando por mais entradas
    rl.close();
  });

```

Múltiplas Perguntas em Sequência

Para fazer várias perguntas em sequência, envolva `rl.question()` em uma Promise e use `async/await` para evitar o *callback hell*.

```

// cadastro.js
const readline = require('readline');
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

// Função auxiliar que transforma rl.question em uma Promise
function pergunta(rl, texto) {
  return new Promise((resolve) => {
    rl.question(texto, (resposta) => {
      resolve(resposta.trim()); // .trim() remove espaços extras
    });
  });
}

// Função principal usando async/await para código mais legível
async function coletarDados() {
  const nome = await pergunta(rl, "Nome completo: ");
  const matricula = await pergunta(rl, "Matrícula: ");
  const cursoStr = await pergunta(rl, "Curso (CC/SI/ES): ");
  const notaStr = await pergunta(rl, "Nota (0-10): ");

  const nota = parseFloat(notaStr.replace(',', '.')); // aceita vírgula ou ponto
  if (isNaN(nota) || nota < 0 || nota > 10) {
    console.error("Nota inválida. Deve ser um número entre 0 e 10.");
    rl.close();
    return;
  }

  const situacao = nota >= 7 ? "Aprovado"
    : nota >= 5 ? "Recuperação"
    : "Reprovado";

  console.log("\n--- Cadastro Registrado ---");
  console.log("Nome: " + nome);
  console.log("Matrícula: " + matricula);
  console.log("Curso: " + cursoStr.toUpperCase());
  console.log("Nota: " + nota.toFixed(1));
  console.log("Situação: " + situacao);
  rl.close();
}

coletarDados().catch((err) => {
  console.error("Erro inesperado:", err.message);
  rl.close();
});

```

Código 53. Múltiplas perguntas com readline usando `async/await`.

Por que transformar `rl.question` em Promise? O método `rl.question()` original usa *callbacks*. Para fazer múltiplas perguntas em sequência com callbacks,

o código fica aninhado e difícil de ler (o *callback hell*). Ao envolver em uma *Promise* e usar *async/await*, o código fica sequencial e legível, mesmo sendo assíncrono. Esse padrão de “promisificar” funções baseadas em *callback* é muito comum no ecossistema Node.js.

Loop de Perguntas com *readline*

Para criar programas interativos que fazem perguntas repetidamente até o usuário decidir sair, combine o padrão *Promise* com um laço *while*.

```
// calculadora.js aceita operações até o usuário digitar 'sair'
const readline = require('readline');
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

function pergunta(texto) {
  return new Promise((resolve) => rl.question(texto, resolve));
}

async function calculadora() {
  console.log("Calculadora simples. Digite 'sair' para encerrar.");
  console.log("Formato: numero operador numero (ex: 10 + 3)\n");

  while (true) {
    const entrada = await pergunta("> ");
    if (entrada.toLowerCase() === "sair") {
      console.log("Até mais!");
      break;
    }

    const partes = entrada.trim().split(/\s+/);
    if (partes.length !== 3) {
      console.log("Formato inválido. Use: numero operador numero");
      continue;
    }

    const a = parseFloat(partes[0]);
    const op = partes[1];
    const b = parseFloat(partes[2]);

    if (isNaN(a) || isNaN(b)) {
      console.log("Erro: os valores devem ser números.");
      continue;
    }

    let resultado;
    switch (op) {
      case '+': resultado = a + b; break;
      case '-': resultado = a - b; break;
      case '*': resultado = a * b; break;
      case '/':
        if (b === 0) { console.log("Erro: divisão por zero."); continue; }
        resultado = a / b;
        break;
      default:
        console.log("Operador inválido. Use: + - * /");
        continue;
    }
    console.log("= " + resultado);
  }
  rl.close();
}

calculadora();
```

Código 54. Calculadora interativa em Node.js com loop de perguntas.

Lendo Linhas de um Arquivo com readline

O módulo `readline` não serve apenas para entrada interativa — ele é igualmente eficiente para processar arquivos de texto linha a linha, sem carregar o arquivo inteiro na memória de uma vez.

```
// processar-arquivo.js
const readline = require('readline');
const fs       = require('fs');

const rl = readline.createInterface({
  input:      fs.createReadStream('notas.csv'),
  crlfDelay: Infinity // trata \r\n (Windows) como uma única quebra de linha
});

let linhaAtual = 0;
const alunos  = [];

// O evento 'line' é emitido para cada linha do arquivo
rl.on('line', (linha) => {
  linhaAtual++;
  if (linhaAtual === 1) return; // pula o cabeçalho do CSV

  const colunas = linha.split(',');
  if (colunas.length < 3) return; // pula linhas malformadas

  const [nome, matricula, nota] = colunas;
  alunos.push({
    nome:      nome.trim(),
    matricula: matricula.trim(),
    nota:      parseFloat(nota)
  });
});

// O evento 'close' é emitido quando o arquivo termina
rl.on('close', () => {
  console.log("Total de alunos processados:", alunos.length);
  const media = alunos.reduce((acc, a) => acc + a.nota, 0) / alunos.length;
  console.log("Média da turma:", media.toFixed(2));
  const aprovados = alunos.filter(a => a.nota >= 7);
  console.log("Aprovados:", aprovados.length);
});
```

Código 55. Leitura de arquivo CSV linha a linha com `readline`.

8.6. Variáveis de Ambiente com `process.env`

Variáveis de ambiente são uma forma padrão de passar configurações para programas sem embutir valores no código-fonte. São essenciais para separar configurações de desenvolvimento e produção (portas, senhas, chaves de API, URLs de banco de dados, etc.).

```
// Lendo variáveis de ambiente
console.log(process.env.PATH);    // PATH do sistema
console.log(process.env.HOME);    // diretório home do usuário
console.log(process.env.NODE_ENV); // "development", "production", "test"

// Usar variável com valor padrão (operador ??)
const porta = process.env.PORT ?? 3000;
const debug = process.env.DEBUG === 'true'; // converte string para boolean

// No início do seu arquivo principal (index.js ou app.js):
require('dotenv').config();

// Agora process.env contém os valores do .env
console.log(process.env.DB_HOST); // "localhost"
console.log(process.env.PORT);    // "3000"
```

Código 56. Lendo e utilizando variáveis de ambiente com `process.env`.

Código 57. Usando o pacote `dotenv` para carregar variáveis de um arquivo `.env`.

⚠ **Nunca versione o arquivo .env no git.** O .env contém senhas, chaves de API e outros dados sensíveis. Adicione-o ao .gitignore imediatamente. Em seu lugar, mantenha um arquivo .env.example com as variáveis necessárias mas sem os valores reais, para que outros desenvolvedores saibam quais variáveis precisam configurar:

```
# .env.example (este sim vai para o git)
PORT=3000
DB_HOST=localhost
DB_NAME=
DB_PASSWORD=
NODE_ENV=development
```

9. Comparativo Rápido e Boas Práticas

9.1. Quando Usar Cada Ambiente

Tarefa	Navegador	Node.js
Experimentar sintaxe JS rapidamente	Console	REPL (node)
Manipular o DOM de uma página	Console	N/A
Testar seleção de elementos CSS	Console	N/A
Depurar um script .js em execução	DevTools	node -inspect
Executar um script de automação	N/A	node script.js
Criar um servidor HTTP	N/A	Node.js + Express
Processar arquivos do sistema	N/A	Node.js + módulo fs
Ler entrada do usuário interativamente	prompt() / formulário HTML	readline
Testar código que será usado no back-end	N/A	Node.js

Tabela 4. Guia de decisão: navegador ou Node.js?

9.2. Fluxo de Trabalho Recomendado para Iniciantes

1. Instale o Node.js (<https://nodejs.org> — versão LTS).
2. Instale o VS Code com as extensões: ESLint, Prettier e Live Server.
3. Para exercícios de lógica pura (sem DOM): escreva .js e execute com node arquivo.js.
4. Para exercícios com interface: crie index.html + app.js, abra com Live Server, use o console do DevTools.
5. Use o console do navegador para inspecionar variáveis e testar expressões rápidas.
6. Use console.log() como ferramenta primária de depuração — coloque-os nos pontos de interesse.
7. Quando o código crescer, use o depurador do VS Code ou o DevTools com breakpoints.

💡 **console.log() é a melhor amiga do iniciante.** Não existe técnica de depuração mais rápida e universal do que adicionar console.log() nos pontos estratégicos do código. Ao identificar um bug, imprima o valor das variáveis relevantes antes e depois da operação suspeita. Isso revela em que momento o valor se torna inesperado. Remova os logs de depuração antes de entregar ou publicar o código.

9.3. Depuração com o Depurador do Node.js

Para problemas mais complexos, o console.log() pode não ser suficiente. O Node.js possui um depurador integrado que pode ser conectado ao VS Code ou ao Chrome

DevTools, permitindo inspecionar variáveis, definir *breakpoints* e executar o código passo a passo.

```
# Executar com o depurador ativo
node --inspect index.js
# Abra o Chrome e navegue para: chrome://inspect
# Clique em "inspect" sob o processo Node.js listado
```

```
# Alternativa: pausar na primeira linha
node --inspect-brk index.js
```

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Executar arquivo atual",
      "program": "${file}"
    },
    {
      "type": "node",
      "request": "launch",
      "name": "Iniciar servidor",
      "program": "${workspaceFolder}/index.js"
    }
  ]
}
```

Código 58. Ativando o depurador do Node.js para uso com o Chrome DevTools.

Código 59. Configuração do depurador do VS Code (.vscode/launch.json).

Resumo dos Pontos-Chave

- **Variáveis:** prefira `const`, use `let` quando necessário, evite `var` em código novo.
 - **Tipos:** JavaScript é dinamicamente tipado; use `===` para comparações; evite depender da coerção implícita.
 - **Funções:** são valores de primeira classe; *arrow functions* têm sintaxe concisa e herdam o `this` léxico; *closures* permitem encapsular estado.
 - **Arrays:** prefira métodos funcionais (`map`, `filter`, `reduce`) a laços imperativos; preste atenção à diferença entre métodos que mutam e que retornam novos arrays.
 - **Objetos:** desestruturação e *spread operator* tornam o código mais expressivo; `Object.entries()` é útil para iterar sobre propriedades.
 - **Callbacks:** são a base da programação assíncrona em JavaScript; entendê-los é pré-requisito para `Promises` e `async/await`.
 - **Dois ambientes:** navegador (DOM, eventos, interface) e Node.js (sistema de arquivos, rede, terminal). Mesma sintaxe, APIs diferentes.
 - **Console do navegador:** abra com `F12` > `Console`. Use `console.log/error/warn/table()`. REPL interativo para experimentação. `$` e `$$` como atalhos para `querySelector`.
 - **prompt/confirm/alert:** funções do navegador para entrada rápida. São síncronas e bloqueantes. Use apenas em protótipos e exercícios — nunca em produção.
 - **Formulário HTML (correto):** leitura real de dados usa elementos HTML + `addEventListener`. Não bloqueante e customizável.
 - **Node.js no terminal:** execute com `node arquivo.js`. REPL com `node` sem argumentos. `process.argv` para argumentos da linha de comando.
 - **readline:** módulo nativo para leitura interativa no terminal. Envolver em `Promise` e use `async/await` para evitar *callback hell*.
 - **process.env:** variáveis de ambiente para configuração. Use `dotenv` para carregar `.env`. Nunca versionar o `.env` no git.
-

Referências e Leituras Complementares

- **MDN Web Docs — JavaScript Reference:** <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript>
- **Eloquent JavaScript (3. ed.) — Marijn Haverbeke:** <https://eloquentjavascript.net/> (gratuito online)
- **You Don't Know JS (série) — Kyle Simpson:** <https://github.com/getify/You-Dont-Know-JS> (gratuito no GitHub)
- **JavaScript.info:** <https://javascript.info/> — tutorial moderno e detalhado, disponível em português
- **ECMAScript Specification:** <https://tc39.es/ecma262/> — especificação oficial da linguagem
- **Node.js Documentation:** <https://nodejs.org/en/docs> — referência oficial para desenvolvimento server-side
- **MDN — Console API:** <https://developer.mozilla.org/pt-BR/docs/Web/API/console>
- **MDN — Window.prompt():** <https://developer.mozilla.org/pt-BR/docs/Web/API/Window/prompt>
- **Node.js — Documentação do readline:** <https://nodejs.org/api/readline.html>
- **Node.js — process.argv e process.env:** <https://nodejs.org/api/process.html>
- **Chrome DevTools — Console:** <https://developer.chrome.com/docs/devtools/console>
- **Pacote dotenv:** <https://www.npmjs.com/package/dotenv>